



دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

داده‌ساختارها و الگوریتم‌ها

جلسه نهم – آرایه و مقدمات لیست پیوندی

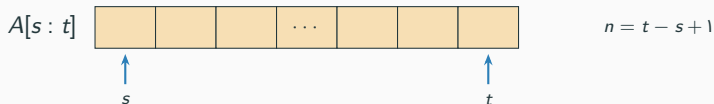
دکتر شهرام گلزاری

پاییز ۱۴۰۵

در پایان این جلسه انتظار داریم که:

- نشانی عنصر $A[i]$ را حساب کنیم و توضیح دهیم چرا دسترسی به آن $O(1)$ است.
- اندیس عنصر $M[i, j]$ را در ترتیب سطری و ترتیب ستونی حساب کنیم.
- توضیح دهیم اشاره‌گر چیست و دنبال کردن آن چه گام‌هایی دارد.
- با یک شکل نشان دهیم گره‌های لیست پیوندی در حافظه پخش‌اند و این چه اثری بر هزینه دسترسی و درج دارد.
- اجرای INSERT-AFTER و DELETE-AFTER را روی یک لیست پیوندی یک‌سویه دنبال کنیم.

آرایه



برای شناختن یک آرایه این‌ها را باید بدانیم:

- **کران‌های اندیس:** اندیس نخستین عنصر s و اندیس آخرین عنصر t .
- **نوع عنصر** (element type): همه‌ی عنصرها از **یک نوع** اند.
- **اندازه‌ی نوع عنصر** (size of element type): هر عنصر b بایت جا می‌گیرد.
- **خانه‌های اشغال‌شده** (occupied): همه‌ی خانه‌ها **پشت سر هم** در حافظه‌اند.
- **نشانی شروع** a : نشانی نخستین بایت $A[s]$ در حافظه.

«فرض می‌کنیم که، مانند بیشتر زبان‌های برنامه‌نویسی، یک آرایه به صورت دنباله‌ای پیوسته از بایت‌ها در حافظه ذخیره می‌شود. اگر نخستین عنصر آرایه اندیس s داشته باشد (برای مثال، در آرایه‌ای با اندیس‌گذاری از 1 ، $s = 1$)، آرایه از نشانی حافظه‌ی a آغاز شود، و هر عنصر آرایه b بایت اشغال کند، آنگاه عنصر i ام بایت‌های $a + b(i - s)$ تا $a + b(i - s + 1) - 1$ را اشغال می‌کند.»

CLRS, 4th ed.، صفحه ۲۵۲ (ترجمه ماشینی)

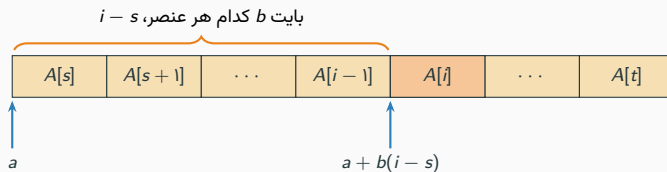
بیشتر آرایه‌های کتاب CLRS از ۱ و چند آرایه از ۰ اندیس‌گذاری شده‌اند. با جای‌گذاری s در فرمول کلی، فرمول‌ها ساده‌تر می‌شوند:

اندیس نخستین عنصر	نخستین بایت عنصر i ام	آخرین بایت عنصر i ام
s (حالت کلی)	$a + b(i - s)$	$a + b(i - s + 1) - 1$
$s = 1$	$a + b(i - 1)$	$a + bi - 1$
$s = 0$	$a + bi$	$a + b(i + 1) - 1$

در اندیس‌گذاری از ۰ فرمول از همه ساده‌تر است: نشانی عنصر i ام فقط $a + bi$ است.

بر اساس *CLRS, 4th ed.*، صفحه ۲۵۲

نخستین بایت $A[i]$ در کجای حافظه است؟ پیش از $A[i]$ دقیقاً $i - s$ عنصر قرار دارد و هر کدام b بایت است.



$$\text{addr}(A[i]) = a + b(i - s)$$

مثال: آرایه‌ی $A[۱۰ : ۳]$ از نوع `int`

فرض کنید هر `int` چهار بایت باشد ($b = ۴$)، $s = ۳$ ، و آرایه از نشانی $a = ۱۰۰۰$ شروع شود.

۱۰۰۰	۱۰۰۴	۱۰۰۸	۱۰۱۲	۱۰۱۶	۱۰۲۰	۱۰۲۴	۱۰۲۸
$A[۳]$	$A[۴]$	$A[۵]$	$A[۶]$	$A[۷]$	$A[۸]$	$A[۹]$	$A[۱۰]$

• تعداد عنصرها: $n = ۱۰ - ۳ + ۱ = ۸$ (روی هم $۸ \times ۴ = ۳۲$ بایت)

• نشانی $A[۷]$: $۱۰۰۰ + ۴(۷ - ۳) = ۱۰۱۶$ (بایت‌های ۱۰۱۶ تا ۱۰۱۹)

• نشانی $A[۱۰]$: $۱۰۰۰ + ۴(۱۰ - ۳) = ۱۰۲۸$

برای پیدا کردن $A[۷]$ لازم نشد از $A[۳]$ تا $A[۶]$ را ببینیم، **مستقیم** حساب کردیم.

چرا دسترسی به $A[i]$ از $O(1)$ است؟

- $A[1]$ در آرایه‌ی 10^6 عنصری: ۴ گام
- $A[10^6]$ در آرایه‌ی 10^6 عنصری: باز هم ۴ گام

دسترسی به $Arrayname[i]$ از $O(1)$ است.

برای خواندن $A[i]$:

گام	کار
۱	یک تفریق: $i - s$
۲	یک ضرب: $b(i - s)$
۳	یک جمع: $a + \dots$
۴	یک بار خواندن از حافظه در آن نشانی

تعداد گام‌ها همیشه ۴ است، نه به i بستگی دارد و نه به n .

$O(1)$ بودن دسترسی روی سه پایه ایستاده است:

۱. خانه‌ها پشت سر هم اند، پس نشانی را می‌توان با یک فرمول حساب کرد.
۲. عنصرها هم‌اندازه‌اند، پس b ثابت است و فرمول ساده می‌ماند.
۳. حافظه دسترسی تصادفی (random access) دارد، پس رفتن به هر نشانی زمان ثابت دارد.

در ادامه‌ی جلسه می‌بینیم وقتی پایه‌ی نخست را کنار بگذاریم (لیست پیوندی) چه به دست می‌آوریم و چه از دست می‌دهیم.

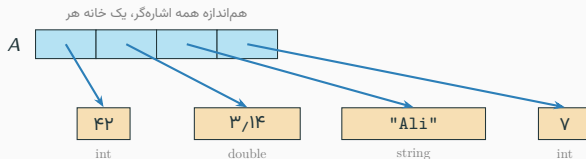
اگر عناصر هم‌اندازه نباشند؟

«(...) اگر عناصر یک آرایه بتوانند تعداد بایت‌های متفاوتی اشغال کنند، فرمول‌های بالا به کار نمی‌آیند، زیرا اندازه‌ی عنصر b ثابت نیست. در چنین مواردی، عناصر آرایه معمولاً اشیائی با اندازه‌های متفاوت‌اند و آنچه در واقع در هر عنصر آرایه قرار می‌گیرد یک اشاره‌گر به آن شیء است.»

CLRS, 4th ed.، صفحه ۲۵۲ (ترجمه ماشینی)

اندازه‌ی اشاره‌گر به اندازه‌ی شیء بستگی ندارد. پس فرمول نشانی، نشانی اشاره‌گر را می‌دهد و برای رسیدن به خود شیء باید اشاره‌گر را دنبال کرد.

بر اساس *CLRS, 4th ed.*، صفحه ۲۵۲



آرایه‌های چندبعدی

«ماتریس یا آرایه‌ی دوبعدی را معمولاً با یک یا چند آرایه‌ی یک‌بعدی نمایش می‌دهیم. دو شیوه‌ی رایج‌تر برای ذخیره‌ی یک ماتریس، ترتیب سطری و ترتیب ستونی است. (...) در ترتیب سطری، ماتریس سطر به سطر ذخیره می‌شود، و در ترتیب ستونی، ماتریس ستون به ستون ذخیره می‌شود.»

(ترجمه ماشینی) *CLRS, 4th ed.*، صفحه ۲۵۳

$M =$

۱	۲	۳	row 1
۴	۵	۶	row 2

ماتریس 3×2 مثال کتاب (رابطه‌ی ۱.۱۰)

- ترتیب سطری (row-major order): سطرها پشت سر هم.
- ترتیب ستونی (column-major order): ستونها پشت سر هم.
- ماتریس $m \times n$ یعنی m سطر و n ستون.

ذخیره‌ی ماتریس در یک آرایه‌ی یک‌بعدی

ترتیب سطری، دو سطر ۱ ۲ ۳ و ۴ ۵ ۶ را پشت سر هم می‌گذارد و ترتیب ستونی سه ستون ۱ ۴ و ۲ ۵ و ۳ ۶ را:

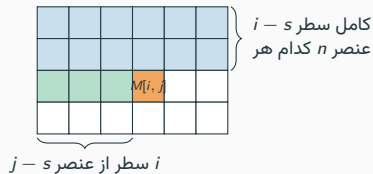


رنگ هر خانه سطر آن در ماتریس M را نشان می‌دهد. در ترتیب ستونی دو عنصر یک سطر دیگر کنار هم نیستند.

بر اساس *CLRS, 4th ed.*، صفحه ۲۵۳ (شکل ۱.۱۰ الف و ب)

فرض: سطرها، ستون‌ها و آرایه‌ی یک‌بعدی همه از s اندیس‌گذاری شده‌اند. در ترتیب سطری، پیش از $M[i, j]$ چند عنصر آمده است؟

- سطرهای s تا $i - 1$ کامل آمده‌اند: $n(i - s)$ عنصر.
- در سطر i ، ستون‌های s تا $j - 1$ آمده‌اند: $j - s$ عنصر.
- اندیس آرایه از s شروع می‌شود، پس s را اضافه می‌کنیم.



ترتیب ستونی: $s + m(j - s) + (i - s)$

ترتیب سطری: $s + n(i - s) + (j - s)$

در ترتیب ستونی نقش سطر و ستون عوض می‌شود: پیش از $M[i, j]$ تعداد $j - s$ ستون کامل، هر کدام با m عنصر، آمده است.

فرمول‌ها بر اساس *CLRS, 4th ed.*، صفحه ۲۵۳

ترتیب ستونی	ترتیب سطری	
$s + m(j - s) + (i - s)$	$s + n(i - s) + (j - s)$	حالت کلی
$i + m(j - 1)$	$n(i - 1) + j$	$s = 1$
$i + mj$	$ni + j$	$s = 0$

مثال: در ماتریس M با اندیس‌گذاری از ۱ ($m = 2$ و $n = 3$)، عنصر $M[2, 1] = 4$:

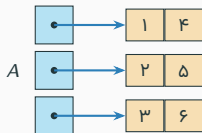
• ترتیب سطری: اندیس $4 = 3(2 - 1) + 1$ (آرایه‌ی $\langle 1, 2, 3, 4, 5, 6 \rangle$)

• ترتیب ستونی: اندیس $2 = 2 + 2(1 - 1)$ (آرایه‌ی $\langle 1, 4, 2, 5, 3, 6 \rangle$)

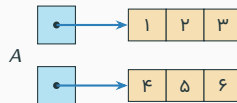
با فرمول نشانی آرایه‌ی یک‌بعدی ترکیب کنیم: اگر آرایه از نشانی a آغاز شود و هر عنصر b بایت باشد، در ترتیب سطری
 $\text{addr}(M[i, j]) = a + b(n(i - s) + (j - s))$ ، که باز هم با تعداد ثابتی عمل حساب می‌شود.

بر اساس *CLRS, 4th ed.*، صفحه ۲۵۳

آرایه‌ی آبی A اشاره‌گر نگه می‌دارد و هر اشاره‌گر به آرایه‌ی یک سطر (یا یک ستون) اشاره می‌کند:



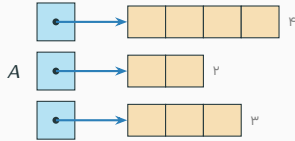
(ت) ستونی: n آرایه‌ی ستون به طول m
 $A[j][i] = M[i, j]$



(پ) سطری: m آرایه‌ی سطر به طول n
 $A[i][j] = M[i, j]$ به سطر i اشاره می‌کند و $A[j][i] = M[i, j]$

دسترسی به $M[i, j]$ دو گام دارد: خواندن اشاره‌گر $A[i]$ و سپس خواندن خانه‌ی j از آرایه‌ای که به آن اشاره می‌کند.

بر اساس *CLRS, 4th ed.*, صفحه‌های ۲۵۳ و ۲۵۴ (شکل ۱.۱۰ پ و ت)



یک آرایه‌ی ناهموار با سطرهایی به طول ۴، ۲ و ۳

بر اساس *CLRS, 4th ed.*، صفحه ۲۵۴

- نمایش **تک‌آرایه‌ای** روی ماشین‌های امروزی معمولاً **کارا تر** است.
- نمایش **چندآرایه‌ای** گاهی **انعطاف بیشتری** دارد.
- مثلاً اجازه‌ی **آرایه‌های ناهموار** (ragged arrays) را می‌دهد: در نسخه‌ی سطری سطرها می‌توانند طول‌های متفاوت داشته باشند (و در نسخه‌ی ستونی، ستون‌ها).

در نمایش بلوکی (block representation)، ماتریس به بلوک‌هایی تقسیم می‌شود و هر بلوک پیوسته ذخیره می‌شود. مثال: ماتریس 4×4 با بلوک‌های 2×2 :

۱	۲	۳	۴
۵	۶	۷	۸
۹	۱۰	۱۱	۱۲
۱۳	۱۴	۱۵	۱۶

۱	۲	۵	۶	۳	۴	۷	۸	۹	۱۰	۱۳	۱۴	۱۱	۱۲	۱۵	۱۶
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----

ترتیب ذخیره: $\langle ۱, ۲, ۵, ۶, ۳, ۴, ۷, ۸, ۹, ۱۰, ۱۳, ۱۴, ۱۱, ۱۲, ۱۵, ۱۶ \rangle$ ، یعنی چهار عنصر هر بلوک کنار هم.

بر اساس *CLRS, 4th ed.*، صفحه ۲۵۴

آرایه‌ی مرتب

آرایه‌ی $A[1 : n]$ مرتب است اگر

$$A[1] \leq A[2] \leq \dots \leq A[n]$$

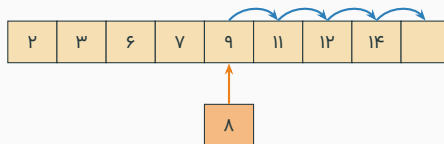
مثلاً خروجی MERGE-SORT در جلسه‌های پیش:

۱	۲	۳	۴	۵	۶	۷	۸
۲	۳	۶	۷	۹	۱۱	۱۲	۱۴

مرتب بودن چه چیزی به ما می‌دهد؟

- کمینه $A[1]$ و بیشینه $A[n]$ است، پس یافتن آن‌ها $\Theta(1)$ است به جای $\Theta(n)$.
- مقدارهای تکراری کنار هم‌اند، پس با یک پیمایش پیدا می‌شوند.
- جست‌وجو با جست‌وجوی دودویی در $\Theta(\lg n)$ به جای $\Theta(n)$ انجام می‌شود.

می‌خواهیم ۸ را در آرایه‌ی مرتب درج کنیم و آرایه مرتب بماند:



- جای ۸ بین ۷ و ۹ است، پس ۹، ۱۱، ۱۲ و ۱۴ باید یک خانه به راست بروند.
 - چرا؟ چون خانه‌ها پشت سر هم اند، نمی‌توان بین دو خانه جا باز کرد.
 - بدترین حالت (درج کوچک‌ترین مقدار): هر n عنصر جابه‌جا می‌شوند، پس هزینه $\Theta(n)$ است.
- ویژگی پیوستگی که دسترسی به هر عنصر را $O(1)$ می‌کند، درج و حذف را پرهزینه می‌کند.

اشاره گر

در آرایه **ترتیب منطقی** عناصرها همان **ترتیب مکانی** آنها در حافظه است: $A[i+1]$ دقیقاً پس از $A[i]$ قرار دارد.

آنچه پیوستگی می‌گیرد

برای درج یا حذف در میانه باید عناصرها را جابه‌جا کرد، پس $\Theta(n)$ است.

آنچه پیوستگی می‌دهد

نشانی هر عنصر با فرمول حساب می‌شود، پس دسترسی $O(1)$ است.

ایده: ترتیب را از مکان جدا کنیم. هر عنصر هر جای حافظه که باشد، **نشانی عنصر بعدی** را همراه خودش نگه دارد.

برای نگه داشتن یک نشانی به متغیری نیاز داریم که مقدارش نشانی باشد: **اشاره‌گر**.

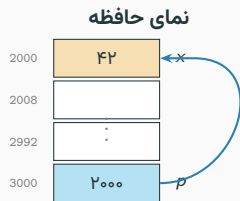
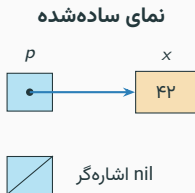
حافظه: دنباله‌ای از خانه‌های شماره‌دار

- حافظه را می‌توان دنباله‌ی بسیار بلندی از بایت‌ها دید. هر بایت یک شماره دارد که **نشانی** (address) آن است.
- هر متغیر چند بایت پشت سر هم را می‌گیرد و نشانی متغیر، نشانی **نخستین بایت** آن است.



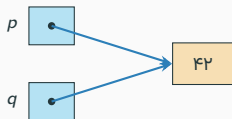
همین دید پایه‌ی فرمول $\text{addr}(A[i]) = a + b(i - s)$ بود: آرایه یک تکه‌ی پیوسته از این بایت‌هاست.

اشاره‌گر (pointer) متغیری است که مقدارش نشانی یک خانه‌ی دیگر حافظه است.



- به جای نوشتن عدد نشانی، از خانه‌ی اشاره‌گر یک پیکان به خانه‌ای که نشانی‌اش را دارد می‌کشیم.
- مقدار ویژه‌ی `nil` (در جاوا: `null`) یعنی اشاره‌گر به هیچ خانه‌ای اشاره نمی‌کند. آن را با خط مورب نشان می‌دهیم.

دو اشاره‌گر به یک خانه



پس از $p = q$ ، هر دو نشانی ۲۰۰۰ را دارند. خود خانه‌ی ۴۲ کپی نشده است و تغییر آن از راه q ، از راه p هم دیده می‌شود.

در لیست پیوندی، تغییر یک اشاره‌گر یعنی تغییر ساختار، بدون جابه‌جا کردن داده‌ها.

برای خواندن مقداری که p به آن اشاره می‌کند:

گام	کار
۱	خواندن p : نشانی ۲۰۰۰
۲	خواندن خانه‌ی ۲۰۰۰: مقدار ۴۲

هر دو گام با فرض دسترسی تصادفی زمان ثابت دارند، پس دنبال کردن یک اشاره‌گر $O(1)$ است.

اندازه‌ی اشاره‌گر به اندازه‌ی چیزی که به آن اشاره می‌کند بستگی ندارد (مثلاً در ماشین‌های ۶۴ بیتی معمولاً ۸ بایت).

در جاوا عدد نشانی را نمی‌بینیم. متغیری از نوع یک کلاس، **ارجاع** (reference) به یک شیء را نگه می‌دارد که همان نقش اشاره‌گر را دارد و null یعنی ارجاعی به هیچ شیء. جدول زیر دستوره‌ای شبکه‌کد کتاب و معادل جاوای آن‌ها را نشان می‌دهد:

CLRS pseudocode	Java
$x = \text{ALLOCATE-NODE}()$	<code>x = new Node()</code>
$x = \text{ALLOCATE-NODE}(e, n)$	<code>x = new Node(element e, next n)</code>
$\text{FREE-NODE}(x)$	<code>delete x</code>
$x.\text{next}$	<code>x.getNext()</code>
$x.\text{next} = n$	<code>x.setNext(n)</code>

Allocate-Node جای یک گره تازه را **هر جای آزاد حافظه** که باشد می‌گیرد و نشانی آن را برمی‌گرداند.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۳، با نمادگذاری CLRS

لیست پیوندی

«در حالت کلی، یک لیست تعدادی عنصر را با ترتیب مشخصی ذخیره می‌کند. در این لیست عنصر اول، عنصر بعد و قبل از هر عنصر، و اغلب، عنصر آخر مشخص و قابل دسترسی هستند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۱

اعمال معمول روی لیست:

- ایجاد لیست تهی و محاسبه‌ی اندازه‌ی لیست،
- درج یک عنصر در ابتدا یا انتها، یا پس از یک عنصر داده‌شده،
- حذف عنصر نخست، یا حذف عنصر پس از یک عنصر داده‌شده.

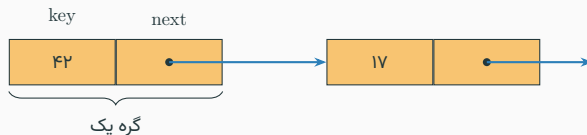
بسته به کاربرد، اعمال دیگری مانند مرتب‌سازی، ادغام دو لیست مرتب و حذف عناصر تکراری هم اضافه می‌شوند. لیست را می‌توان به صورت پیوندی، یک‌سویه، حلقه‌ای، دوسویه، سلسله‌مراتبی یا در حالت کلی پیاده‌سازی کرد.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۱۳۱ و ۱۳۲

گره: داده و اشاره‌گر به گره بعدی

«هر عنصر یک لیست ساده‌ی یک‌سویه به نام `node`، دارای دو مؤلفه‌ی `element` و `next` است. مؤلفه‌ی `element` حاوی تمام اطلاعاتی است که می‌خواهیم در آن عنصر ذخیره کنیم. (...) مؤلفه‌ی `next` هر عنصر **آدرس عنصر بعدی** است و تنها از این طریق می‌توان به آن عنصر دسترسی داشت.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۲



مؤلفه‌ی `next` همان اشاره‌گری است که دیدیم: گره خودش نمی‌داند کجای حافظه است، فقط می‌داند گره بعدی کجاست.

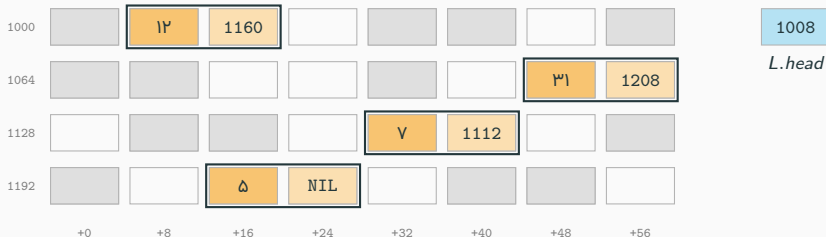
در CLRS مؤلفه‌ی داده‌ی گره `key` نامیده می‌شود. از این پس به جای `element` همین نام را به کار می‌بریم.

بر اساس *CLRS, 4th ed.*، بخش ۲.۱۰

گره‌ها در حافظه پخش‌اند

لیست $\langle ۱۲, ۷, ۳۱, ۵ \rangle$ در حافظه. **فیلدهای یک گره در یک بلوک‌اند:** هر قاب تیره یک گره با دو خانه‌ی key و next است. خانه‌های خاکستری را داده‌های دیگر برنامه گرفته‌اند.

مدل ساده‌شده

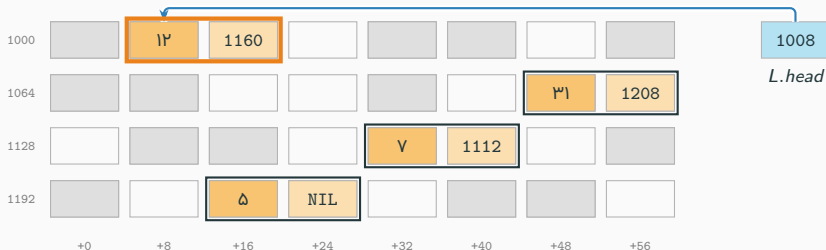


گره‌ها به ترتیب لیست در حافظه نیامده‌اند و میانشان داده‌های دیگری هست. ترتیب را فقط خانه‌های next مشخص می‌کنند.

گره‌ها در حافظه پخش‌اند

لیست $\langle ۱۲, ۷, ۳۱, ۵ \rangle$ در حافظه. **فیلدهای یک گره در یک بلوک‌اند:** هر قاب تیره یک گره با دو خانه‌ی key و next است. خانه‌های خاکستری را داده‌های دیگر برنامه گرفته‌اند.

مدل ساده‌شده

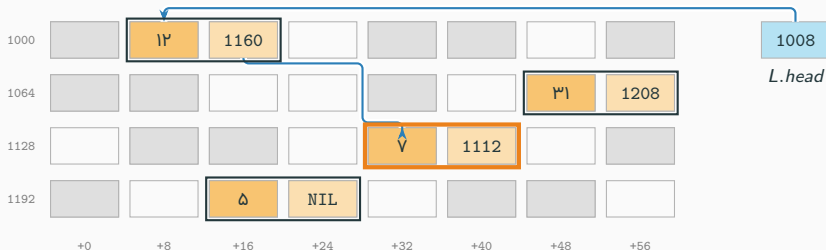


پیمایش از **L.head** آغاز می‌شود: مقدار آن، ۱۰۰۸، نشانی گره نخست (۱۲) است.

گره‌ها در حافظه پخش‌اند

لیست $\langle ۱۲, ۷, ۳۱, ۵ \rangle$ در حافظه. **فیلدهای یک گره در یک بلوک‌اند:** هر قاب تیره یک گره با دو خانه‌ی key و next است. خانه‌های خاکستری را داده‌های دیگر برنامه گرفته‌اند.

مدل ساده‌شده

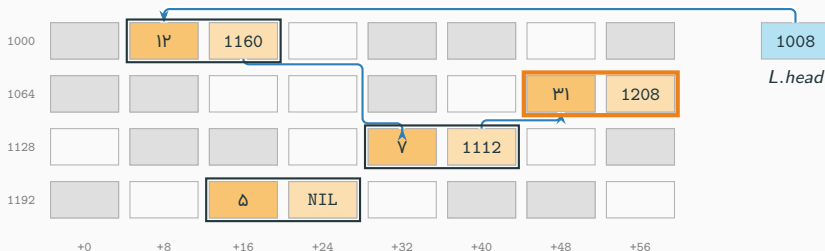


خانه‌ی next گره ۱۲ مقدار ۱۱۶۰ را دارد، پس گره بعدی (۷) در نشانی ۱۱۶۰ است.

گره‌ها در حافظه پخش‌اند

لیست $\langle ۱۲, ۷, ۳۱, ۵ \rangle$ در حافظه. **فیلدهای یک گره در یک بلوک‌اند:** هر قاب تیره یک گره با دو خانه‌ی key و next است. خانه‌های خاکستری را داده‌های دیگر برنامه گرفته‌اند.

مدل ساده‌شده

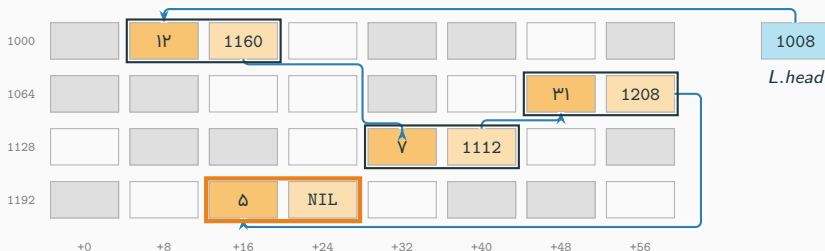


از گره ۷ به نشانی ۱۱۱۲ می‌رویم که در حافظه **پیش از** گره ۷ است. ترتیب منطقی به مکان در حافظه بستگی ندارد.

گره‌ها در حافظه پخش‌اند

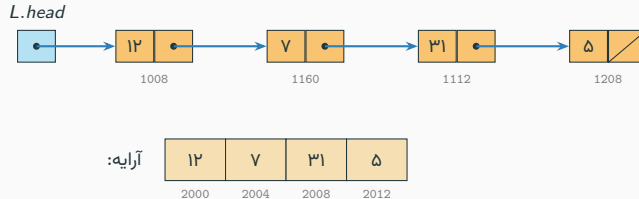
لیست $\langle ۱۲, ۷, ۳۱, ۵ \rangle$ در حافظه. **فیلدهای یک گره در یک بلوک‌اند:** هر قاب تیره یک گره با دو خانه‌ی key و next است. خانه‌های خاکستری را داده‌های دیگر برنامه گرفته‌اند.

مدل ساده‌شده



گره ۳۱ به ۱۲۰۸ اشاره می‌کند. next گره ۵ برابر nil است، پس لیست همین‌جا تمام می‌شود.

همان لیست را معمولاً به شکل زنجیره می‌کشیم، اما نشانی واقعی گره‌ها **نه پشت سر هم اند و نه صعودی**:



- در **آرایه**، مکان ترتیب را تعیین می‌کند: عنصر بعدی همیشه b بایت جلوتر است.
- در **لیست پیوندی**، اشاره‌گرها ترتیب را تعیین می‌کنند: گره بعدی می‌تواند هر جای حافظه باشد.
- بهای این آزادی: برای هر عنصر یک اشاره‌گر اضافه در حافظه.

- گره‌ها پشت سر هم نیستند، پس **فرمولی برای نشانی** گره k ام وجود ندارد.
- نشانی گره k ام فقط در مؤلفه‌ی `next` گره $(k - 1)$ ام نوشته شده است.
- پس باید از گره نخست، $k - 1$ اشاره‌گر را **یکی‌یکی** دنبال کنیم.

دسترسی به گره k ام: $\Theta(k)$ و در بدترین حالت $\Theta(n)$
(در آرایه: $O(1)$)

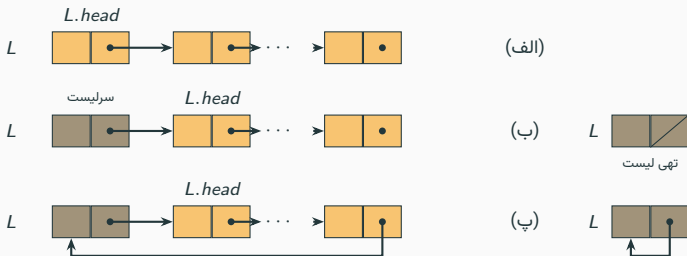
`GET( $L, k$ )`

```
1  $x = L.head$   
2 for  $i = 1$  to  $k - 1$   
3    $x = x.next$   
4 return  $x$ 
```

(رویه‌ی کمکی خود درس، با نمادگذاری CLRS)

گاهی در ابتدای لیست یک گره اضافه به نام **سرلیست** (header) یا (sentinel) می‌گذاریم تا حالت‌های خاص لیست تهی پیش نیاید.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۳



(الف) لیست پیوندی بدون سرلیست، (ب) لیست پیوندی با سرلیست، (پ) لیست پیوندی حلقه‌ای با سرلیست.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۴ (بازترسیم شکل ۴-۱، با نمادگذاری CLRS)

فرض: x مقدار یک عنصر و n یک عنصر (گره) از لیست است. مؤلفه‌ی $L.size$ تعداد عناصر و $L.head$ نخستین عنصر لیست L را برمی‌گرداند.

- $CREATE-LIST(L)$: ساختن لیست تهی L
- $SIZE(L)$: تعداد عناصر لیست
- $FIRST(L)$: نخستین عنصر لیست
- $IS-EMPTY(L)$: آیا لیست تهی است؟
- $INSERT-FIRST(L, x)$: درج x در ابتدای لیست
- $INSERT-AFTER(L, x, n)$: درج x پس از عنصر n
- $DELETE-FIRST(L)$: حذف نخستین عنصر
- $DELETE-AFTER(L, n)$: حذف عنصر پس از n (اگر وجود داشته باشد)

درج و حذف پس از یک گره تعریف شده‌اند، چون در لیست یک‌سویه از هر گره فقط به گره بعدی راه داریم.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۱۳۳ و ۱۳۴

INSERT-FIRST(L, x)

- 1 $L.head = \text{ALLOCATE-NODE}(x, \text{FIRST}(L))$
- 2 $L.size = L.size + 1$

INSERT-AFTER(L, x, n)

- 1 **if** $n == \text{NIL}$
- 2 **then error** element is empty
- 3 $n.next = \text{ALLOCATE-NODE}(x, n.next)$
- 4 $L.size = L.size + 1$

• هیچ حلقه‌ای در رویه‌ها نیست و هر کدام تعداد ثابتی دستور دارد.

CREATE(L)

- 1 $L.size = 0$

SIZE(L)

- 1 **return** $L.size$

FIRST(L)

- 1 **if** $\text{SIZE}(L) \neq 0$
- 2 **then return** $L.head$
- 3 **else error** list is empty

ISEMPTY(L)

- 1 **return** $\text{SIZE}(L) == 0$

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۵، با نمادگذاری *CLRS*

- خط ۴ رویه‌ی DELETE-AFTER گره r را از زنجیره دور می‌زند: n مستقیم به گره پس از r اشاره می‌کند.
- خط ۳ لازم است: بدون r ، پس از خط ۴ دیگر نشانی گره حذف‌شده را نداریم تا آن را با FREE-NODE آزاد کنیم.
- ترتیب دستورها مهم است، چون تغییر یک اشاره‌گر ممکن است تنها راه رسیدن به یک گره را از بین ببرد.

DELETE-FIRST(L)

```

1 if isEmpty( $L$ )
2   then error list is empty
3  $n = \text{FIRST}(L)$ 
4  $L.\text{head} = n.\text{next}$ 
5 FREE-NODE( $n$ )
6  $L.\text{size} = L.\text{size} - 1$ 
```

DELETE-AFTER(L, n)

```

1 if isEmpty( $L$ ) or  $n == \text{NIL}$  or  $n.\text{next} == \text{NIL}$ 
2   then error element does not exist
3  $r = n.\text{next}$ 
4  $n.\text{next} = r.\text{next}$ 
5 FREE-NODE( $r$ )
6  $L.\text{size} = L.\text{size} - 1$ 
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۵، با نمادگذاری

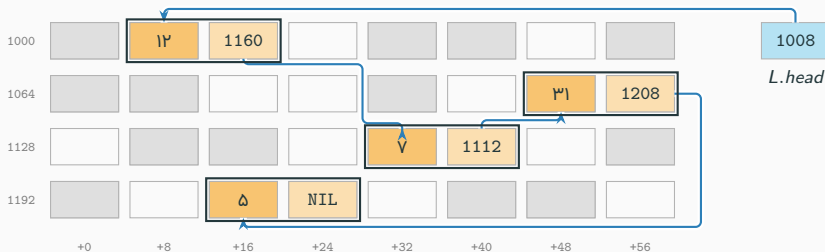
CLRS

INSERT-AFTER در حافظه: ۸ پس از گره ۷

با $x = 8$ و n گره ۷ (نشانی ۱۱۶۰)

$n.next = \text{ALLOCATE-NODE}(x, n.next)$

مدل ساده‌شده



پیش از درج: $n.next = 1112$ ، یعنی گره ۷ به گره ۳۱ اشاره می‌کند.

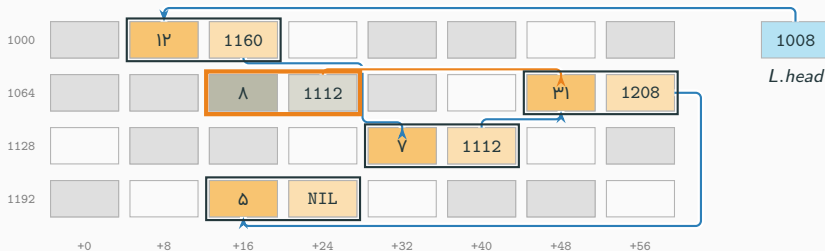
بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۲-۴ ب)

INSERT-AFTER در حافظه: ۸ پس از گره ۷

با $x = 8$ و n گره ۷ (نشانی ۱۱۶۰)

$n.next = \text{ALLOCATE-NODE}(x, n.next)$

مدل ساده‌شده



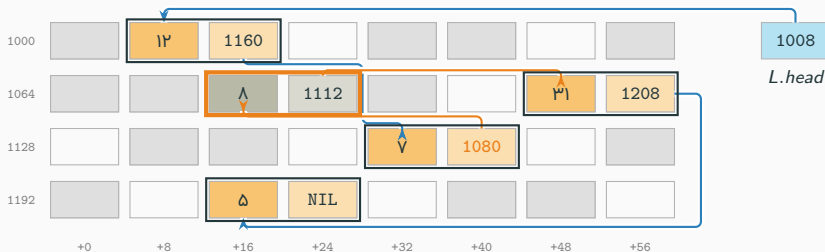
گره تازه را در یک **بلوک آزاد دلخواه** (دو خانه‌ی آزاد کنار هم)، این‌جا نشانی ۱۰۸۰، می‌سازد. مؤلفه‌ی next آن مقدار قبلی $n.next$ ، یعنی ۱۱۱۲، را می‌گیرد.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۲-۴ ب)

INSERT-AFTER در حافظه: درج ۸ پس از گره ۷

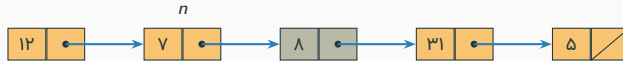
با $x = 8$ و n گره ۷ (نشانی ۱۱۶۰) $n.next = \text{ALLOCATE-NODE}(x, n.next)$

مدل ساده‌شده



نشانی گره تازه در $n.next$ نوشته می‌شود. ترتیب منطقی اکنون $\langle 12, 7, 8, 31, 5 \rangle$ است و هیچ عنصری جابه‌جا نشد: فقط دو خانه‌ی next مقدار گرفتند.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۲-۴ ب)



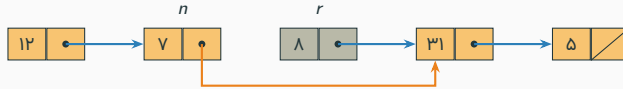
می‌خواهیم گره پس از n (گره ۸) را حذف کنیم.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۲-۴ ت)



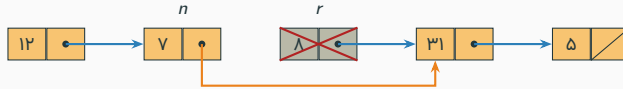
خط ۳: $r = n.next$. نشانی گره ۸ را در r نگه می‌داریم.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۲-۴ ت)



خط ۴: $n.next = r.next$. اکنون n مستقیم به گره ۳۱ اشاره می‌کند و گره ۸ از زنجیره بیرون افتاده است.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۲-۴ ت)



خط ۵: $\text{Free-Node}(r)$. حافظه‌ی گره ۸ آزاد می‌شود. باز هم هیچ عنصری جابه‌جا نشد.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۲-۴ ت)

آرایه در برابر لیست پیوندی یک‌سویه

عمل	آرایه	لیست پیوندی یک‌سویه
دسترسی به عنصر k ام	$O(1)$	$\Theta(k)$
درج پس از یک عنصر داده‌شده	$\Theta(n)$ در بدترین حالت	$O(1)$
حذف پس از یک عنصر داده‌شده	$\Theta(n)$ در بدترین حالت	$O(1)$
درج در ابتدا	$\Theta(n)$	$O(1)$
حافظه‌ی اضافه برای هر عنصر	ندارد	یک اشاره‌گر

- هزینه‌ی $O(1)$ درج و حذف در لیست به شرطی است که گره n **در دست** باشد. اگر ابتدا باید آن را پیدا کنیم، پیدا کردنش $\Theta(k)$ هزینه دارد.
- هیچ‌کدام در همه‌چیز بهتر نیست: انتخاب داده‌ساختار به عمل‌هایی بستگی دارد که بیشتر انجام می‌دهیم.

هزینه‌ی $O(1)$ اعمال لیست بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶

منابع

- Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms* (CLRS), 4th Edition, MIT Press, 2022
- محمد قدسی — داده‌ساختارها و مبانی الگوریتم‌ها، انتشارات فاطمی، چاپ ۱۳۹۷